

LABORATORY OBSERVATION / RECORD

Course	Full Stack Web Development	Experiment No.	Task 5
Course Code	23CM4121	Student Name	R Abhiram
Reg. No.	A24126552114	Date	21-09-2026

1. TITLE

Express.js Student Server with Multiple Routes

2. AIM

To create a basic Express.js server with three routes — /, /students and /about — where /students returns a list of at least five students and /about displays information about the application, using appropriate HTTP methods and routing.

3. OBJECTIVE

The objectives of this experiment are:

- To set up a Node.js project and install the Express.js framework.
- To create an Express application instance and start it on a chosen port.
- To define GET routes that respond to different URL paths.
- To return HTML content from one route and JSON data from another.
- To organise route handlers using Express's routing methods (app.get()).
- To test each route using a browser or an HTTP client such as curl.

4. THEORY

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building web servers and APIs. It simplifies routing, request handling and response generation compared to using Node's built-in http module directly.

- **express():** the top-level function exported by the express module; calling it creates an Express application.
- **app.get(path, handler):** registers a route handler that runs when an HTTP GET request is made to the given path.
- **req (request) object:** represents the incoming HTTP request, including URL, headers, query and route parameters.
- **res (response) object:** used to send data back to the client, e.g. res.send() for text/HTML or res.json() for JSON.
- **res.send():** sends a response of various types (string, HTML, Buffer); Express sets the Content-Type automatically.
- **res.json():** sends a JSON response and automatically sets the Content-Type header to application/json.
- **app.listen(port, callback):** binds and starts the server, listening for connections on the specified port.
- **Routing:** the way an application responds to a client request for a specific endpoint (URL path and HTTP method).

Flow: client sends an HTTP GET request to a path → Express matches the path to a registered route → the route handler runs → a response (HTML or JSON) is sent back to the client.

5. ALGORITHM / PROCEDURE

1. Initialise a Node.js project (npm init) and install Express (npm install express).
2. Import express and create an application instance using express().
3. Define an in-memory array of at least five student objects (id, name, course, year).
4. Define a GET route for '/' that sends a welcome HTML message.
5. Define a GET route for '/students' that responds with the students array as JSON using res.json().
6. Define a GET route for '/about' that sends HTML describing the application.
7. Start the server with app.listen() on a chosen port (3000) and log a confirmation message.
8. Run the server with node server.js and test each route in a browser or with curl.
9. Verify the response body and status code for every route.

6. PROGRAM

server.js

```
// Express.js Student Server
const express = require('express');
const app = express();
const PORT = 3000;

// Sample list of at least five students
const students = [
  { id: 1, name: 'Aarav Sharma', course: 'Full Stack Web Development', year: 2 },
  { id: 2, name: 'Diya Patel', course: 'Data Science', year: 3 },
  { id: 3, name: 'Rohan Mehta', course: 'Full Stack Web Development', year: 2 },
  { id: 4, name: 'Sneha Reddy', course: 'Artificial Intelligence', year: 1 },
  { id: 5, name: 'Kabir Nair', course: 'Cyber Security', year: 4 },
  { id: 6, name: 'Ishita Rao', course: 'Full Stack Web Development', year: 2 }
];

// Route 1: Home route
app.get('/', (req, res) => {
  res.send('<h1>Welcome to the Student Management Server</h1><p>Visit /students or /about</p>');
});

// Route 2: Students route - returns list of students
app.get('/students', (req, res) => {
  res.json(students);
});

// Route 3: About route - application information
app.get('/about', (req, res) => {
  res.send(
    '<h2>About This Application</h2>' +
    '<p>This is a basic Express.js server built for Task 7 of the Full Stack Web Development ' +
    'course. It demonstrates routing using the GET HTTP method with three endpoints: ' +
    '/, /students and /about.</p>'
  );
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

7. CODE EXPLANATION

Code	Explanation
<code>require('express')</code>	Imports the Express framework.
<code>express()</code>	Creates an Express application instance, <code>app</code> .
<code>students</code> array	In-memory list of six student objects used as sample data for the <code>/students</code> route.
<code>app.get('/', handler)</code>	Registers the home route; responds with a welcome HTML message.
<code>app.get('/students', handler)</code>	Registers the students route; responds with the students array.
<code>res.json(students)</code>	Serialises the students array to JSON and sets Content-Type: <code>application/json</code> .
<code>app.get('/about', handler)</code>	Registers the about route; responds with HTML describing the app.
<code>app.listen(PORT, cb)</code>	Starts the HTTP server on port 3000 and runs the callback once it is ready.

8. TEST CASES AND EXECUTION DATA

The server was started with `node server.js` and each route was tested using `curl`, observing the HTTP status, content type and response body.

Test Case	Route	Method	Expected Result	Actual Result	Status
TC-01	<code>/</code>	GET	Welcome HTML message, status 200	Welcome message displayed, 200 OK	Pass
TC-02	<code>/students</code>	GET	JSON array of 6 students, status 200	JSON array of 6 students returned, 200 OK	Pass
TC-03	<code>/about</code>	GET	About HTML message, status 200	About message displayed, 200 OK	Pass
TC-04	<code>/students</code>	GET (headers)	Content-Type: <code>application/json</code>	Content-Type: <code>application/json; charset=utf-8</code>	Pass
TC-05	<code>/unknown</code>	GET	404 Not Found (no matching route)	404 Not Found returned by Express	Pass

9. OUTPUT

Output 1: Starting the server

```
$ node server.js
Server is running at http://localhost:3000
```

Output 2: TC-01 – GET /

```
$ curl -s http://localhost:3000/  
<h1>Welcome to the Student Management Server</h1><p>Visit /students or /about</p>
```

Output 3: TC-02 – GET /students

```
$ curl -s http://localhost:3000/students  
[  
  {"id":1,"name":"Aarav Sharma","course":"Full Stack Web Development","year":2},  
  {"id":2,"name":"Diya Patel","course":"Data Science","year":3},  
  {"id":3,"name":"Rohan Mehta","course":"Full Stack Web Development","year":2},  
  {"id":4,"name":"Sneha Reddy","course":"Artificial Intelligence","year":1},  
  {"id":5,"name":"Kabir Nair","course":"Cyber Security","year":4},  
  {"id":6,"name":"Ishita Rao","course":"Full Stack Web Development","year":2}  
]
```

The route returned all six student records as a JSON array, satisfying the "at least five students" requirement.

Output 4: TC-03 – GET /about

```
$ curl -s http://localhost:3000/about  
<h2>About This Application</h2><p>This is a basic Express.js server built for  
Task 7 of the Full Stack Web Development course. It demonstrates routing using  
the GET HTTP method with three endpoints: /, /students and /about.</p>
```

Output 5: TC-04 – response headers for /students

```
$ curl -s -D - -o /dev/null http://localhost:3000/students  
HTTP/1.1 200 OK  
X-Powered-By: Express  
Content-Type: application/json; charset=utf-8  
Content-Length: 432  
ETag: W/"1b0-8WwCVzOw6ngip/rRzzKSD5AZRkg"  
Connection: keep-alive
```

Express automatically set the Content-Type header to application/json for the res.json() response.

10. RESULT

Thus, a basic Express.js server was successfully created and tested with three routes. The / route returned a welcome HTML message, the /students route returned a JSON array of six students, and the /about route returned HTML information about the application. All routes were verified using curl, confirming correct status codes, content types and response bodies.